is never going to come. It's therefore undesirable to block on external input from a thread that also performs tasks that other threads may be waiting for.

That briefly covers unwanted blocking. What about race conditions?

10.1.2. Race conditions

Race conditions are the most common cause of problems in multithreaded code—many deadlocks and livelocks only actually manifest because of a race condition. Not all race conditions are problematic—a race condition occurs anytime the behavior depends on the relative scheduling of operations in separate threads. A large number of race conditions are entirely benign; for example, which worker thread processes the next task in the task queue is largely irrelevant. However, many concurrency bugs are due to race conditions. In particular, race conditions often cause the following types of problems:

- *Data races*—A data race is the specific type of race condition that results in undefined behavior because of unsynchronized concurrent access to a shared memory location. I introduced data races in chapter 5 when we looked at the C++ memory model. Data races usually occur through incorrect usage of atomic operations to synchronize threads or through access to shared data without locking the appropriate mutex.
- *Broken invariants*—These can manifest as dangling pointers (because another thread deleted the data being accessed), random memory corruption (due to a thread reading inconsistent values resulting from partial updates), and double-free (such as when two threads pop the same value from a queue, and so both delete some associated data), among others. The invariants being broken can be temporal- as well as value-based. If operations on separate threads are required to execute in a particular order, incorrect synchronization can lead to a race condition in which the required order is sometimes violated.
- *Lifetime issues*—Although you could bundle these problems in with broken invariants, this really is a separate category. The basic problem with bugs in this category is that the thread outlives the data that it accesses, so it is accessing data that has been deleted or otherwise